

5.5.3 Compiler optimizations

Basic compilation techniques can generate inefficient code. Compilers use a wide range of algorithms to optimize the code they generate.

Inlining

Function inlining replaces a subroutine call to a function with equivalent code to the function body. By substituting the function call's parameters into the body, the compiler can generate a copy of the code that performs the same operations but without the subroutine overhead. C++ provides an *inline* qualifier that allows the compiler to substitute an inline version of the function. In C, programmers can perform inlining manually or by using a preprocessor macro to define the code body.

Outlining is the opposite operation to inlining—a set of similar sections of code replaced with calls to an equivalent function. Although inlining eliminates function call overhead, it also increases program size. Inlining also inhibits sharing of the function code in the cache—because the inlined copies are distinct pieces of code, they cannot be represented by the same code in the cache. Outlining is sometimes useful to improve the cache behavior of common functions.

Loop transformations

Loops are important program structures—although they are compactly described in the source code, they often use a large fraction of the computation time. Many techniques have been designed to optimize loops.

A simple but useful transformation is known as **loop unrolling**, illustrated in the next example. Loop unrolling is important because it helps expose parallelism that can be used by later stages of the compiler.

Example 5.4 Loop Unrolling

Here is a simple C loop:

```
for (i = 0; i < N; i++) {
    a[i] = b[i] * c[i];
}
```

This loop is executed a fixed number of times, namely, N . A straightforward implementation of the loop would create and initialize the loop variable i , update its value on every iteration, and test it to see whether to exit the loop. However, because the loop is executed a fixed number of times, we can generate more direct code.

If we let $N = 4$, then we can substitute this straight-line code for the loop:

```
a[0] = b[0] * c[0];
a[1] = b[1] * c[1];
a[2] = b[2] * c[2];
a[3] = b[3] * c[3];
```

This unrolled code has no loop overhead code at all, that is, no iteration variable and no tests. But the unrolled loop has the same problems as the inlined procedure—it may interfere with the cache and expands the amount of code required.

We do not, of course, have to fully unroll loops. Rather than unroll the above loop four times, we could unroll it twice. Unrolling produces this code:

```
for (i = 0; i < 2; i++) {
    a[i*2] = b[i*2]*c[i*2];
    a[i*2 + 1] = b[i*2 + 1]*c[i*2 + 1];
}
```

In this case, because all operations in the two lines of the loop body are independent, later stages of the compiler may be able to generate code that allows them to be executed efficiently on the CPU's pipeline.

Loop fusion combines two or more loops into a single loop. For this transformation to be legal, two conditions must be satisfied. First, the loops must iterate over the same values. Second, the loop bodies must not have dependencies that would be violated if they are executed together—for example, if the second loop's i th iteration depends on the results of the $i + 1$ th iteration of the first loop, the two loops cannot be combined. **Loop distribution** is the opposite of loop fusion, that is, decomposing a single loop into multiple loops.

Dead code elimination

Dead code is code that can never be executed. Dead code can be generated by programmers, either inadvertently or purposefully. Dead code can also be generated by compilers. Dead code can be identified by **reachability analysis**—finding the other statements or instructions from which it can be reached. If a given piece of code cannot be reached, or it can be reached only by a piece of code that is unreachable from the main program, then it can be eliminated. **Dead code elimination** analyzes code for reachability and trims away dead code.

Register allocation

Register allocation is a very important compilation phase. Given a block of code, we want to choose assignments of variables (both declared and temporary) to registers to minimize the total number of required registers.

The next example illustrates the importance of proper register allocation.

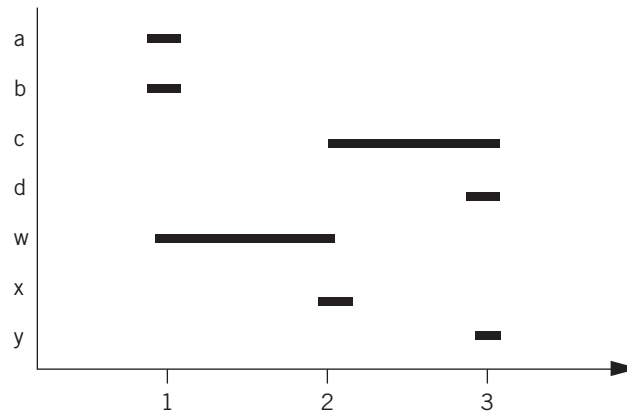
Example 5.5 Register Allocation

To keep the example small, we assume that we can use only four of the ARM's registers. In fact, such a restriction is not unthinkable—programming conventions can reserve certain registers for special purposes and significantly reduce the number of general-purpose registers available.

Consider this C code:

```
w = a + b; /* statement 1 */
x = c + w; /* statement 2 */
y = c + d; /* statement 3 */
```

A naive register allocation, assigning each variable to a separate register, would require seven registers for the seven variables in the above code. However, we can do much better by reusing a register once the value stored in the register is no longer needed. To understand how to do this, we can draw a **lifetime graph** that shows the statements on which each statement is used. Here is a lifetime graph in which the x axis is the statement number in the C code and the y axis shows the variables.



A horizontal line stretches from the first statement where the variable is used to the last use of the variable; a variable is said to be **live** during this interval. At each statement, we can determine every variable currently in use. The maximum number of variables in use at any statement determines the maximum number of registers required. In this case, statement two requires three registers: *c*, *w*, and *x*. This fits within the four-register limitation. By reusing registers once their current values are no longer needed, we can write code that requires no more than four registers. Here is one register assignment:

a	r0
b	r1
c	r2
d	r0
w	r3
x	r0
y	r3

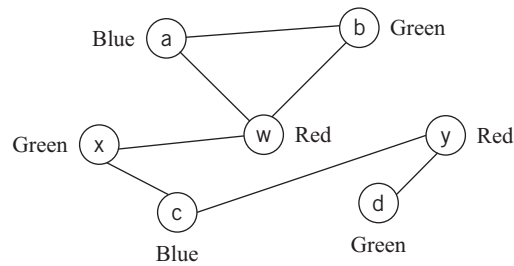
Here is the ARM assembly code that uses the above register assignment:

```

LDR r0,[p_a]      ; load a into r0 using pointer to a (p_a)
LDR r1,[p_b]      ; load b into r1
ADD r3,r0,r1      ; compute a + b
STR r3,[p_w]      ; w = a + b
LDR r2,[p_c]      ; load c into r2
ADD r0,r2,r3      ; compute c + w, reusing r0 for x
STR r0,[p_x]      ; x = c + w
LDR r0,[p_d]      ; load d into r0
ADD r3,r2,r0      ; compute c + d, reusing r3 for y
STR r3,[p_y]      ; y = c + d

```

If a section of code requires more registers than are available, we must **spill** some of the values out to memory temporarily. After computing some values, we write the values to temporary memory locations, reuse those registers in other computations,

**FIGURE 5.17**

Using graph coloring to solve the problem of Example 5.5.

and then reread the old values from the temporary locations to resume work. Spilling registers is problematic in several respects: it requires extra CPU time and uses up both instruction and data memory. Putting effort into register allocation to avoid unnecessary register spills is worth your time.

We can solve register allocation problems by building a **conflict graph** and solving a graph coloring problem. As shown in Fig. 5.17, each variable in the high-level language code is represented by a node. An edge is added between two nodes if they both live at the same time. The graph coloring problem is to use the smallest number of distinct colors to color all the nodes such that no two nodes are directly connected by an edge of the same color. The figure shows a satisfying coloring that uses three colors. Graph coloring is NP-complete, but there are efficient heuristic algorithms that can give good results on typical register allocation problems.

Lifetime analysis assumes that we have already determined the order in which we will evaluate operations. In many cases, we have freedom in the order in which we do things. Consider this expression:

$$(a + b) * (c - d)$$

We have to do the multiplication last, but we can do either the addition or the subtraction first. Different orders of loads, stores, and arithmetic operations may also result in different execution times on pipelined machines. If we can keep values in registers without having to reread them from main memory, we can save execution time and reduce code size as well.

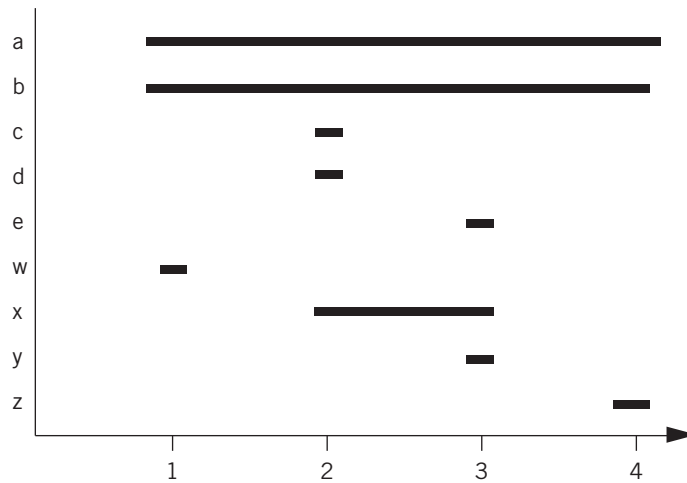
The next example shows how proper **operator scheduling** can improve register allocation.

Example 5.6 Operator Scheduling for Register Allocation

Here is a sample C code fragment:

```
w = a + b; /* statement 1 */
x = c + d; /* statement 2 */
y = x + e; /* statement 3 */
z = a - b; /* statement 4 */
```

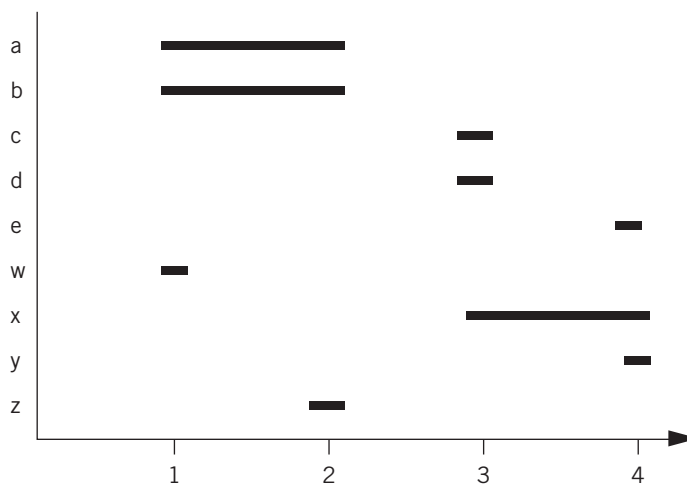
If we compile the statements in the order in which they were written, we get this register graph:



Because *w* is needed until the last statement, we need five registers at statement 3, even though only three registers are needed for the statement at line 3. If we swap statements 3 and 4 (renumbering them 39 and 49), we reduce our requirements to three registers. Here is the modified C code:

```
w = a + b; /* statement 1 */
z = a - b; /* statement 29 */
x = c + d; /* statement 39 */
y = x + e; /* statement 49 */
```

And here is the lifetime graph for the new code:



Compare the ARM assembly code for the two code fragments. We have written both assuming that we have only four free registers. In the *before* version, we do not have to write out any values, but we must read *a* and *b* twice. The *after* version allows us to retain all values in registers as long as we need them.

Before version	After version
LDR r0,a	LDR r0,a
LDR r1,b	LDR r1,b
ADD r2,r0,r1	ADD r2,r1,r0
STR r2,w ; w = a + b	STR r2,w ; w = a + b
LDR r0,c	SUB r2,r0,r1
LDR r1,d	STR r2,z ; z = a - b
ADD r2,r0,r1	LDR r0,c
STR r2,x ; x = c + d	LDR r1,d
LDR r1,e	ADD r2,r1,r0
ADD r0,r1,r2	STR r2,x ; x = c + d
STR r0,y ; y = x + e	LDR r1,e
LDR r0,a ; reload a	ADD r0,r1,r2
LDR r1,b ; reload b	STR r0,y ; y = x + e
SUB r2,r1,r0	
STR r2,z ; z = a - b	

Scheduling

We have some freedom to choose the order in which operations will be performed. We can use this to our advantage—for example, we may be able to improve the register allocation by changing the order in which operations are performed, thereby changing the lifetimes of the variables.

We can solve scheduling problems by keeping track of resource utilization over time. We do not have to know the exact microarchitecture of the CPU—all we have to know is that, for example, instruction types 1 and 2 both use resource A while instruction types 3 and 4 use resource B. CPU manufacturers generally disclose enough information about the microarchitecture to allow us to schedule instructions even when they do not provide a detailed description of the CPU's internals.

We can keep track of CPU resources during instruction scheduling using a **reservation table** [Kog81]. As illustrated in Fig. 5.18, rows in the table represent instruction execution time slots and columns represent resources that must be scheduled. Before scheduling an instruction to be executed at a particular time, we check the reservation table to determine whether all resources needed by the instruction are available at that time. Upon scheduling the instruction, we update the table to note all resources used by that instruction. Various algorithms can be used for the scheduling itself, depending on the types of resources and instructions involved, but the reservation table provides a good summary of the state of an instruction scheduling problem in progress.

Time	Resource A	Resource B
t	X	
t + 1	X	X
t + 2	X	
t + 3		X

FIGURE 5.18

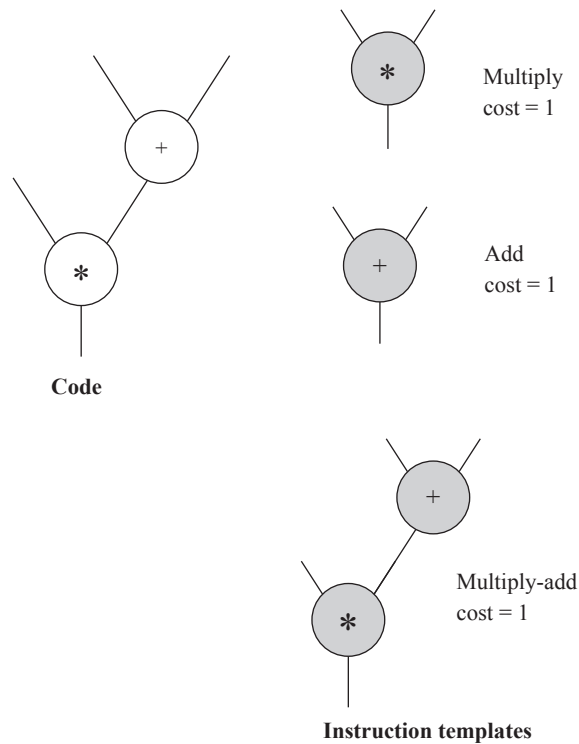
A reservation table for instruction scheduling.

We can also schedule instructions to maximize performance. As we know from Section 3.6, when an instruction that takes more cycles than normal to finish is in the pipeline, pipeline bubbles appear that reduce performance. **Software pipelining** is a technique for reordering instructions across several loop iterations to reduce pipeline bubbles. Some instructions take several cycles to complete; if the value produced by one of these instructions is needed by other instructions in the loop iteration, then they must wait for that value to be produced. Rather than pad the loop with no-ops, we can start instructions from the next iteration. The loop body then contains instructions that manipulate values from several different loop iterations—some of the instructions are working on the early part of iteration $n + 1$, others are working on iteration n , and still others are finishing iteration $n - 1$.

Instruction selection

Selecting the instructions to use to implement each operation is not trivial. There may be several different instructions that can be used to accomplish the same goal, but they may have different execution times. Moreover, using one instruction for one part of the program may affect the instructions that can be used in adjacent code. Although we cannot discuss all the problems and methods for code generation here, a little bit of knowledge helps us envision what the compiler is doing.

One useful technique for generating code is **template matching**, illustrated in Fig. 5.19. We have a DAG that represents the expression for which we want to generate code. To be able to match up instructions and operations, we represent instructions using the same DAG representation. We shaded the instruction template nodes to distinguish them from code nodes. Each node has a cost, which may be simply the execution time of the instruction or may include factors for size, power consumption, and so on. In this case, we have shown that each instruction takes the same amount of time, and thus all have a cost of 1. Our goal is to cover all nodes in the code DAG with instruction DAGs—until we have covered the code DAG we have not generated code for all the operations in the expression. In this case, the lowest-cost covering uses the multiply-add instruction to cover both nodes. If we first tried to cover the bottom node with the multiply instruction, we would find ourselves blocked from using the multiply-add instruction. Dynamic programming can be used to efficiently find the lowest-cost covering of trees, and heuristics can extend the technique to DAGs.

**FIGURE 5.19**

Code generation by template matching.

Understanding your compiler

Clearly, the compiler can vastly transform your program during the creation of assembly language. But compilers are also substantially different in terms of the optimizations they perform. Understanding your compiler can help you get the best code out of it.

Studying the assembly language output of the compiler is a good way to learn about what the compiler does. Some compilers will annotate sections of code to help you make the correspondence between the source and assembler output. Starting with small examples that exercise only a few types of statements will help. You can experiment with different optimization levels (the `-O` flag on most C compilers). You can also try writing the same algorithm in several ways to see how the compiler's output changes.

If you cannot get your compiler to generate the code you want, you may need to write your own assembly language. You can do this by writing it from scratch or modifying the output of the compiler. If you write your own assembly code, you must ensure that it conforms to all compiler conventions, such as procedure call linkage. If you modify the compiler output, you should be sure that you have the algorithm